

Applied Cryptography
Week 7/ Zero-Knowledge Proof & Code Obfuscation

Student Name: **Mikheil Davidovi**
Class: **Applied Cryptography**

Submission Date: **November 23, 2025**

GITHUB: <https://github.com/mikheildavidovi-pixel/Sangu2025>

Part 1: Zero-Knowledge Proof (ZKP) Simulation

Authorized:

```
Experiment 1 : 20 / 20  
Average successes: 20.0  
Theoretical per-round success: 0.5  
Probability of passing all 20 rounds: 9.5367431640625e-07
```

Malicious:

```
Running 1 experiment(s), each with 20 rounds.  
Experiment 1: malicious prover succeeded 9/20  
  
Average successes over 1 experiments: 9.000/20  
Empirical per-round success probability: 0.450  
  
Theoretical probabilities:  
Per-round malicious success: 0.5  
Passing all 20 rounds: 0.000000953674316406
```

Based on the experiment, out of 20 rounds, an attacker succeeded 9 times.

The average success probability is 0.450, or 45%.

In a case of 1000 rounds, an average success probability is approximately 0.5

Part 2: Code Obfuscation Challenge

Original Code

```
def factorial(n):
```

```

    if n < 0:
        raise ValueError("n must be non-negative")
    r = 1
    for i in range(2, n + 1):
        r *= i
    return r

if __name__ == "__main__":
    for x in range(6):
        print(x, factorial(x))

```

Manually Obfuscated Code

```

def f(x):
    if x < 0:
        raise Exception
    a = 1
    b = 2
    while b <= x:
        a = a * b
        b = b + 1
        if False and b % 3 == 1:
            a = a
    return a

if __name__ == "__main__":
    y = 0
    while y < 6:
        print(y, f(y))
        y += 1

```

I manually renamed identifiers, flattened control flow, and added dead code. These changes remove meaningful structure, make execution paths less obvious, and hinder human reverse engineering.

Automatically Obfuscated Code

```
# Pyarmor 9.2.0 (trial), 000000, non-profits, 2025-11-23T17:40:24.661329
from pyarmor_runtime_000000 import __pyarmor__
__pyarmor__(__name__, __file__,
b'PY000000\x00\x03\xe\x00+\xe\r\n\x80\x00\x01\x00\x08\x00\x00\x00\x05\x00\x00\x00@\x00\x00\x00k\x04\x00\x00\x12\t\x04\x00\x88\xbc\xcd\x17\x85+biwA3\x11\x91\xb5Z\x00\x00\x00\x00\x00\x00\x00\x00\x8bCSJ\t\xcf5\xfaZ\t\xc1\x0eTL\x86e\x00\x16\xb1\xa0\xde\x08\xc7M\xe6\xe6\xef\xbc\x96@t0\x00\xebBZ]\xb10wcc\xd53\xcf6\xb4h\xbd\b3\xe6\xfe;]0n\x98\x9a\xc5@\x84\x8ex\xde\xcf41E1\x7f\xd16\xe5\xbc6G\xb0T\x03\xf9/w\xcf\xc3.\xd1\xfe\xba*\xc4\x0b\xd0&! \xc0\x83W)\x99\x9eU\xd5k4\xcf6\x96\x1a\xfe:Q\xabN\x1b\xb9q\xflV([?\xf0\x1a\xbe\xbc\x1a\x8fL\rw&nFy\x8f*\x91\xbdI\x00\xe3e\xcf30\x93e\x0c\xa0!7M) \xc5\xa1\xb5\xc5JZ\xbaG\xa6\xae\xcz\xa3\x0f\xa1\x06\xa2\xd1-\x13\xe5n\xb7\x0b)h\xa3\x94\xa1ya\xcf9\x0b)\xdf\xe2>\xc1\xe9c50\x1dn\xcd\xa8\x04\xd2g\xd7\tj\xaa9\x0c\x91\xde\x07\xac3\xf8\xc6\xbb4"\xb2?\xd2\x82E\xc6\xe2\x1f\xcf4ub\x01\x8f\xb0\xfcDV\xe8\x10z\x81\xa7\x87\xe7i\xbeF\xc5\xbb\x85\xed\x90=\xffB\x01\xcb\xff\xfaF\xcb\xba\xb0\xd1\xf2\xef8N\x16\xdf\xba\x86i@\xa6\x94\xe5\xa3\x8e\xfl\x83\xc4\xc6\xe3\xed\xd7/1B\xc2-\xa6\xd8\xcf9\\\xf5*\tj4\xcc\x0c\x94\xcf3\x05\x12\x19d\xab\x82\x06\xc5\x17\xc3\xdc\xff"\x9c\x95\x1fe\xa0\nF\xab\xa6\x1aZ3I\xcf\xc6\x03?\x84\xad6\x89&>\xef\xa6\xd2`Z\x11r/\xae\xef\xcf8@K\x06\x83\xf8\xc5}\xe2\xa5}\xf95\xd8\xec\xacVK\xb4\xfbRb\xa5\x01\x98\x93\x89\x0c\xe5~,#\x98\xfe\xc6\xfe\x12\xd8\xcf#\x93S\x1a\x97\xcf4\xd2Z\x12\x9a\xfc\xfaa\x11\xa5C\xb4\x88\xae\xab3\t\xbd\xcf6&\xc9V\x0e7\xd2\x92~\x12\x95H\xaa\x9eH\xca\xb8\xcar\x948\xe9<sTE\x07xp\x9d\xd8cg{\x92\x8c\x88>\x8c-@\xal\xfc^\xf2\x8e\xc1\x12\xaf\x92Q\x1a\xd5\x8c0\xba"\xc4\x15\xbfsys\xd7\xd2\xe1%Os\x95ri\x8d\x13\xfe\xd7B\x00\x9e\x8b\x95\xceF\xbd\xcd\x9b{\xab\xbc\xe8\x9e\xd1\xd8\x84\xda\xce\' \xb2\xbb\xe7j\xcf2mZ\xc3\xbb0jF\xcf8\x9eV\xd9\x92\xb7g\x88\xa8\xcdg\xcb~i+\xaa\x9f\xc3\xf7|\x1f3\x13\xb8\x7f\xb9\xf3\xe0\xb6g\xe8u\xebn\x8b\xaaax[9V\xbl\xiac:NG\xcf6\xcf7\xa8\x17\x8f3\xc8\x8f\r\b3\x19\xd1\x05qJn\xbl\xa0\xc6\xcf2})_\xa3\xa8\xa1\xbc\xd1\x94\x1f\xb6\x03\\\xcey\xa0Mh\x93u\t\x92\\/\x87\xff\x8aH\b8\xe0\xb4\xd1\xa6:n\bBI\xc2~\x8b\x05\xe7U\xc5\xed\x1e\xdfw\xc6\xe4\x0c1\xbd\xa0\xe5\xcf3\x0e0\xc5C\xbcO\xaf\xbe\xcc\x16\xcf12\xca\x15\x0b\xafEP14\x15\x1b\x9b\xfa@\xf7f\'L\xcaa\x073o\xeb"\xc4\xe4\x85\xcf5m\x8e\xd0\xd9E\xcdw\x139\xcaKr\xcf3\xe2\xa0\xf0\x9e\xcaRGvFAX\xde\x91`%k\x0f\x9bw&\xc8r\xfcI`= \xf2e\r\xd1\x86S\xbehD@\xec\xcb|\xf2q0*\x99
```

```
\x94-j\xfe\x12v\x07M\xba\x9dG\xac\xc2B\xe7\xb09<\x8d=\xe6\x10\xe7\xee\xb4\xb1
\xb6\x98\xdb\xf8\xe3\xaf\xdd\xe2\xa2T\x19\x8c\x10\xe6\xcb\x00(1\x94L\xa7\x85\
xe8d\xd321\x18\x17\x06\xe6\x86qq\xb22\xb8fB\x9c\xcd\x88\x17)nm\xdc\x0c\x0b"\x
e3&\xb6\xf2\xdd6t
\xcd|\x04\x9b!\xa6\xdlpW\xb5c\xd3o\xe6\xd6\xfd?\x9fWV\xfl\xe\xcc\x89z\x01;r\xaf
%v>*q#\xcf\x08\x16\x03\xfd\xed*\xa9\xfb"Y/\xeb\x8br\x91C\xe2\xf4\xef\xb4>|\xf
7\xf1!Y\xb4g\xd8\ryi\x7f\xd7\xb4\xca\x9f\x10\xacO=\x01*\x11r\xa7\x8d\x8b2\x02
\x8c\x08\x8eVC\x82$7\xd8\xab
6E\x95\xc3\x02\x17\xd5\x88\xfc\xbb\xfcph\x85;\xe4\x0c"m\xa6\x05\x1e\xed\x9e\x
de\x10@\xcd-\x833\xd8\x05/\xb3\xe6\xdd!iN\xd4\xf8ma?\x93\x1cF\xce2\xa2)\x1c\x
f0\xe3\xf8g\xc5-\x0b'\x16\x92\\xca\x1ev\x88\x11\xb5\x1c\x125\xd7A>(\x89\xd6
\xf6b\x18\xd4\xfdD\xe9\x89\x19_z\x0f\xe6\x8f\x0c\x8e\x06\x85\xf9\x83\xc2\xe2\
xd6\xfd\x0f\x1\xeb4\xc4\xbad\x98\x135X\x9by\xe0H(\xdf\x95\xb431\x98\x81\x8e\xe7
\'7\x89\x08\xb3\xb3X9\xfd*\xdc\x9c<_6\xe4\x0e\xc1i}S\x1c\x1bKZ\xcfqc\x18\xdbo
\x8aU\xfd\xf4\x9aot\xfe\xb2\xc0\xe2p\xd5\xe8\x89U\xd6\\\xab\x1d\x01\xbd;\x9a(
{\xf42;\x1b\xf2\x01!\xacw\x90\x93\x95\x9dn\x02[P\xd1\x89L\xd6\xbd\xea\xf5] ')
```

I used PyArmor. PyArmor encrypts Python bytecode, renames identifiers, and adds runtime protections. It produces a heavily protected, non-readable output that is much harder to decompile than simple minification.

PyArmor Output:

```
C:\WINDOWS\system32>pyarmor gen C:\Users\Ronin\PycharmProjects\PythonProject\Lab1\obfuscation.py
INFO      Python 3.14.0
INFO      Pyarmor 9.2.0 (trial), 000000, non-profits
INFO      Platform windows.x86_64
INFO      search inputs ...
INFO      find script C:\Users\Ronin\PycharmProjects\PythonProject\Lab1\obfuscation.py
INFO      find 1 top resources
INFO      start to generate runtime files
INFO      target platforms {'windows.amd64'}
INFO      write dist\pyarmor_runtime_000000\pyarmor_runtime.pyd
INFO      generate runtime files OK
INFO      start to obfuscate scripts
INFO      process resource "obfuscation"
INFO      obfuscating file obfuscation.py
INFO      write dist\obfuscation.py
INFO      obfuscate scripts OK
```

Output of an obfuscated file I ran in PyCharm:

```
0 1
1 1
2 2
3 6
4 24
5 120

Process finished with exit code 0
```

Output is similar to the original file.